

DescriptionAccepted x Editorial SolutionsSubmissions

All Submissions

Accepted28 / 28 testcases passed

yousab21 submitted at Jun 27, 2026 19:32

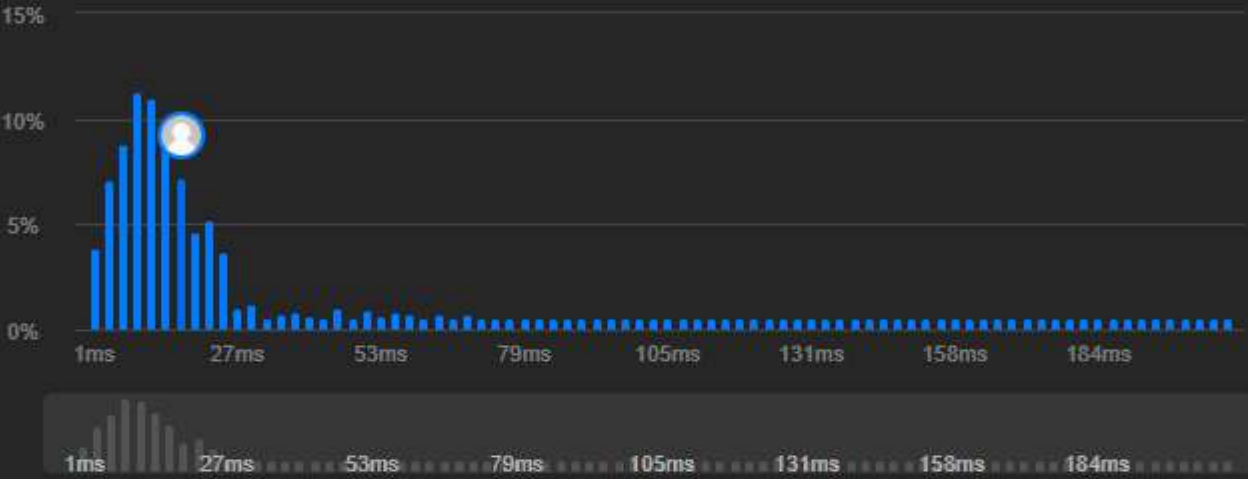
AnalysisSolution

Runtime

16 ms | Beats 52.47%

Memory

48.12 MB | Beats 78.55%



CodeC++

```
1 // C++ already has a built in data structure for hashsets called std::unorde
2 // i will not be using the build in structure as this will trivialize the pr
3 // build one from scratch using normal hash maps
4
5 #include <map>
6
7 class MyHashSet {
8     map<int,int> hashSet;
9
10    MyHashSet() {
11
12    }
13
14    void add(int key) {
15        // map.insert only inserts a key if it doesnt exsis so i dont need to do any checks here
16        // i only care about the key here so i just set the value as 0
17        hashSet.insert({key, 0});
18    }
19
20    void remove(int key) {
21        hashSet.erase(key);
22    }
23
24    bool contains(int key) {
25        // map.find() is a stl map method, it searches for a key and returns an iterator
26        // that pointing to the key if it found it if it doesnt find it it returns an iterator
27        // that points to map.end() which is the address just pass the last element to the map
28        // this is just the agreed upon stl handshake to check if an element exsists in a map
29        auto it = hashSet.find(key);
30        if(it != hashSet.end()){
31            return true;
32        }
33        return false;
34    }
35 };
36
37 /**
38  * Your MyHashSet object will be instantiated and called as such:
39  * MyHashSet* obj = new MyHashSet();
40  * obj->add(key);
41  * obj->remove(key);
42  * bool param_3 = obj->contains(key);
43  */
```

View more

More challenges

706. Design HashMap

1206. Design Skiplist

Write your notes here

Select related tags0/5

✔

Code

C++ v Auto

```
1 // C++ already has a built in data structure for hashsets called std::unordered_set
2 // i will not be using the build in structure as this will trivialize the problem so i will
3 // build one from scratch using normal hash maps
4
5 #include <map>
6
7 class MyHashSet {
8     map<int,int> hashSet;
9     public:
10     MyHashSet() {
11
12     }
13
14     void add(int key) {
15         // map.insert only inserts a key if it doesnt exsis so i dont need to do any checks here
16         // i only care about the key here so i just set the value as 0
17         hashSet.insert({key, 0});
18     }
19
20     void remove(int key) {
21         hashSet.erase(key);
22     }
23
24     bool contains(int key) {
25         // map.find() is a stl map method, it searches for a key and returns an iterator
26         // that pointing to the key if it found it if it doesnt find it it returns an iterator
27         // that points to map.end() which is the address just pass the last element to the map
28         // this is just the agreed upon stl handshake to check if an element exsists in a map
29         auto it = hashSet.find(key);
30         if(it != hashSet.end()){
31             return true;
32         }
33         return false;
34     }
35 };
36
37 /**
38  * Your MyHashSet object will be instantiated and called as such:
39  * MyHashSet* obj = new MyHashSet();
40  * obj->add(key);
41  * obj->remove(key);
42  * bool param_3 = obj->contains(key);
43  */
```

Saved

Ln 14, Col 24

TestcaseTest Result

Accepted

22 / 22 testcases passed

yousab21

submitted at Jun 27, 2026 20:20

Analysis

Solution

Runtime

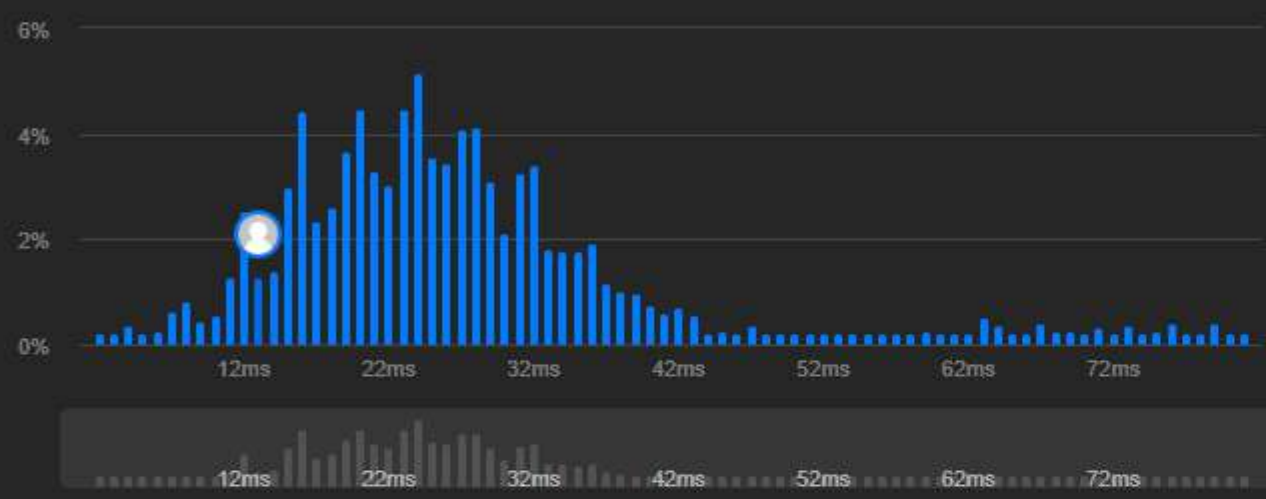
13 ms

Beats 92.66%

Memory

149.17 MB

Beats 61.70%



Code

C++

```
1 class NumMatrix {
2 private:
3     vector<vector<int>>>prefix;
4     void setSizeAndPadding(vector<vector<int>>& matrix){
5         vector<int> row;
6         // if we just fix the prefix array to the same size as the original
7         // with calculating the first row and first column in the prefix usi
8         // the solution is to add a row and a column of zeros as padding so
```

View more

More challenges

- 308. Range Sum Query 2D - Mutable
- 3030. Find the Grid of Region Average

Write your notes here

Select related tags

0/5



Code

C++ v

Auto

```
1 class NumMatrix {
2 private:
3     vector<vector<int>>>prefix;
4     void setSizeAndPadding(vector<vector<int>>& matrix){
5         vector<int> row;
6         // if we just fix the prefix array to the same size as the original we run into problems
7         // with calculating the first row and first column in the prefix using the standard wa
8         // the solution is to add a row and a column of zeros as padding so the actual prefix
9         // calculations doesnt start from the edge (that is why the loops ends with size+1 )
10        for(int i = 0; i< matrix[0].size()+1; i++){
11            row.push_back(0);
12        }
13        for(int i = 0; i< matrix.size()+1; i++){
14            prefix.push_back(row);
15        }
16        return;
17    }
18 public:
19
20     NumMatrix(vector<vector<int>>& matrix) {
21         setSizeAndPadding(matrix);
22         // this is just the classic formula for calculating a 2d prefix sum array
23         for(int i = 1; i< prefix.size();i++){
24             for(int j = 1 ; j< prefix[0].size(); j++){
25                 int prefixElement = matrix[i-1][j-1] + prefix[i][j-1] + prefix[i-1][j] - prefix[i-1][j-1];
26                 prefix[i][j] = prefixElement;
27             }
28         }
29     }
30
31     int sumRegion(int row1, int col1, int row2, int col2) {
32         // new we just use the pre calculated prefix array as follows:
33         // get the sum of the big box until [row2][col2]
34         //substarct the sums of the 2 boxes outside the wanted range
35         // add back the box we substrected twice
36         return prefix[row2+1][col2+1] - prefix[row2+1][col1] - prefix[row1][col2+1] + prefix[row1][col1];
37     }
38 };
39
40 /**
41  * Your NumMatrix object will be instantiated and called as such:
42  * NumMatrix* obj = new NumMatrix(matrix);
43  * int param_1 = obj->sumRegion(row1,col1,row2,col2);
44  */
```

Saved

Ln 36, Col 104

Testcase

Test Result

Accepted 78 / 78 testcases passed

 yousab21 submitted at Jun 27, 2026 21:17

 Analysis

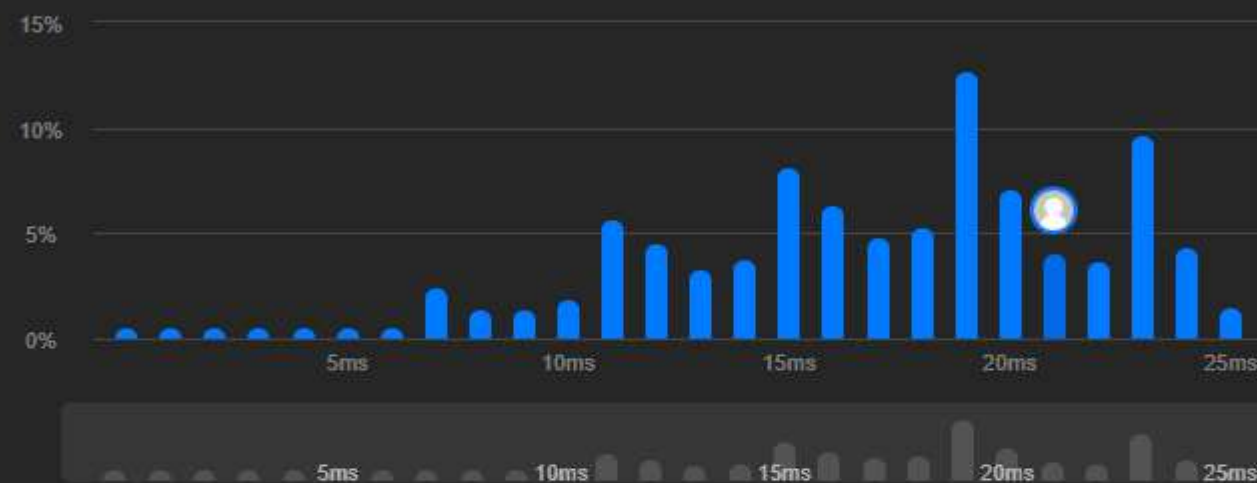
 Solution

 Runtime

21 ms | Beats 29.40%

 Memory

45.74 MB | Beats 70.83%



Code | C++

```
1 class Solution {
2 public:
3     int numRescueBoats(vector<int>& people, int limit) {
4         //the idea is to always pair a light person with a heavy person to n
5         //where only heacy people are left so you have to waste boats
6         //so we start by sorting the array
7         sort(people.begin(), people.end());
8         //the idea of the probelm pretty much screams left and right pointer
```

 View more

More challenges

 691. Stickers to Spell Word

 763. Partition Labels

 3781. Maximum Score After Binary Swaps

Write your notes here

Select related tags

0/5

 Code

C++ Auto

```
1 class Solution {
2 public:
3     int numRescueBoats(vector<int>& people, int limit) {
4         //the idea is to always pair a light person with a heavy person to not reach a point
5         //where only heacy people are left so you have to waste boats
6         //so we start by sorting the array
7         sort(people.begin(), people.end());
8         //the idea of the probelm pretty much screams left and right pointers
9         int left = 0, right = people.size()-1, boatCount =0;
10
11         while(left <= right){
12             //if the heavist person is too heavy to sit with the lightest, he takes a boat alone
13             if(people[right] + people[left] > limit){
14                 right--;
15                 boatCount++;
16             }
17             // if 2 people can sit in a boat together we should pick the lightest and the heaviest pair
18             else{
19                 left++;
20                 right--;
21                 boatCount++;
22             }
23         }
24         return boatCount;
25     }
26 };
```

Saved

Ln 26, Col 3

 Testcase

 Test Result

Accepted

Runtime: 0 ms

 Solution

Runtime
266 ms | Beats 9.04%

Memory
92.92 MB | Beats 14.41%

The chart displays two performance metrics: Runtime and Memory. The Runtime bar chart shows a distribution of execution times, with a peak around 266ms. The Memory bar chart shows a distribution of memory usage, with a peak around 92.92MB. Both charts include a 'Beats' percentage indicating performance relative to other solutions.

View more

0/5

Ln 38, Col 66

Testcase | **Test Result**

Accepted 19 / 19 testcases passed

 yousab21 submitted at Jun 28, 2026 21:08

 Analysis

 Solution



Code | C++

```
1 class Solution {
2 private:
3 // in order to start linking random pointer in the new list we must first co
4 // copied list so here i do a shallow copy leaving all random pointer as nul
5     Node* shallowCopy(Node* head){
6
7         // this is just the standard way to copy a list, i use a dummy node
8         // deal with head edge cases
9
10    }
11
12    }
13 }
```

 View more

More challenges

- 133. Clone Graph

 1485. Clone Binary Tree With Random Pointer
- 1490. Clone N-ary Tree

Write your notes here

Select related tags 0/5

Submit Ctrl Enter

C++ Auto

```
27     Node* equvilantNode(Node* oldHead, Node* newHead, Node* oldNode){
28         if(oldNode == nullptr){
29             return nullptr;
30         }
31
32         // insted i index the list up the node iam traking
33         int index = 0;
34
35         Node* current = oldHead;
36         while(current != oldNode){
37             index++;
38             current = current->next;
39         }
40
41         // i then walk nodes untill that index and return the node i stop on
42         Node* matchingNode = newHead;
43         for(int i = 0; i < index;i++){
44             matchingNode = matchingNode->next;
45         }
46
47         return matchingNode;
48     }
49
50 public:
51     Node* copyRandomList(Node* head) {
52         // first we do the shallow copy
53         Node* copy = shallowCopy(head);
54
55         // now we can walk the list node by node and match random pointers
56         // using the function i built
57         Node* old = head;
58         Node* current = copy;
59         while(current != nullptr && old != nullptr){
60             current->random = equvilantNode(head, copy, old->random);
61             old= old->next;
62             current = current->next;
63         }
64         return copy;
65     }
66 };
```

--INSERT--

Saved

Ln 56, Col 38

 Testcase Test Result

DescriptionAccepted x EditorialSolutionsSubmissions

All Submissions

Accepted 59 / 59 testcases passed

AnalysisSolution

yousab21 submitted at Jun 29, 2026 19:32

Runtime

1 ms | Beats 75.27%

Memory

23.29 MB | Beats 97.01%

Runtime (ms)	Percentage
1ms	25%
2ms	5%
3ms	18%
4ms	17%
5ms	5%
6ms	4%
7ms	8%
8ms	4%

Code C++

```
1 class MyCircularQueue {
2 private:
3 // i will implement it as a linked list here is my node structure
4 struct Node {
5     int val;
6     Node* next;
7     Node(int val){
8         this->val = val;
```

View more

More challenges

641. Design Circular Deque

1670. Design Front Middle Back Queue

Write your notes here

Select related tags 0/5

Code

C++ Auto

```
1 class MyCircularQueue {
2 private:
3 // i will implement it as a linked list here is my node structure
4 struct Node {
5     int val;
6     Node* next;
7     Node(int val){
8         this->val = val;
9         next = nullptr;
10    }
11 };
12
13 // here is the queue attributes + the capacity indicator as the problem wants me
14 // to sed a constant size for the queue
15 int size = 0;
16 int capacity = 0;
17 Node* head = nullptr;
18 Node* tail = nullptr;
19
20 public:
21 MyCircularQueue(int k) {
22     capacity = k;
23 }
24
25 bool enQueue(int value) {
26     // if the queue is not full i create the new node
27     if(isFull()){
28         return false;
29     }
30
31     Node* newNode = new Node(value);
32     if(isEmpty()){
33         // normal edgecase for first node
34         head = tail = newNode;
35     }
36     // put the new node behind the tail
37     else{
38         tail->next = newNode;
39         tail = tail->next;
40     }
41
42     // this is how i link the circular queue, after we are done adding the node
43     // i make the new tail point back to the head making a circle
44     tail->next = head;
```

--INSERT--

Saved

Ln 78, Col 1

Testcase Test Result

DescriptionAccepted x Editorial SolutionsSubmissions

All Submissions

Accepted59 / 59 testcases passed

Analysis

Solution

yousab21

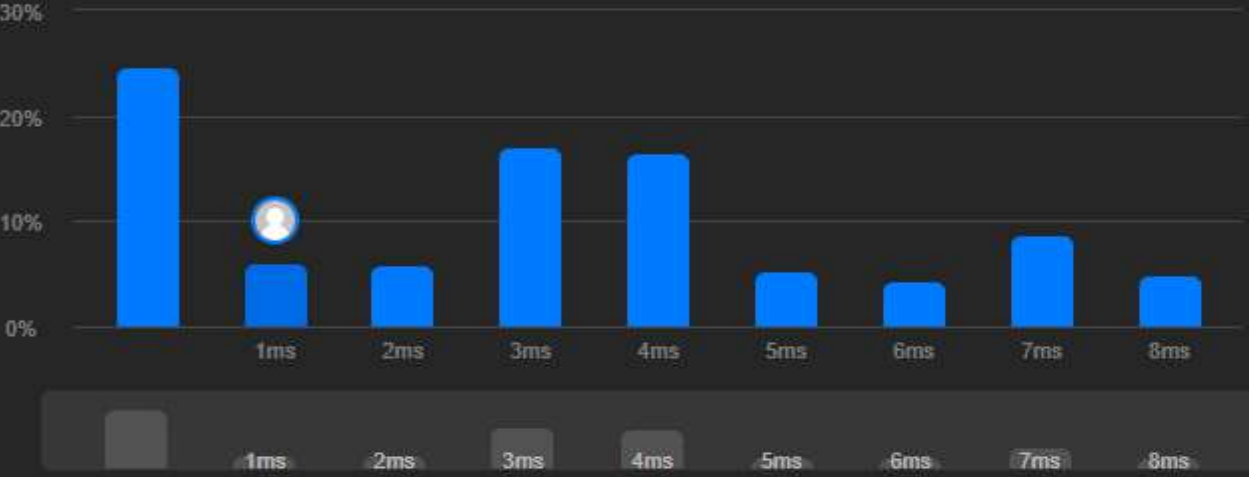
submitted at Jun 29, 2026 19:32

Runtime

1 ms | Beats 75.27%

Memory

23.29 MB | Beats 97.01%



Time Interval	Percentage of Faster Submissions
1ms	~25%
2ms	~6%
3ms	~18%
4ms	~17%
5ms	~6%
6ms	~5%
7ms	~9%
8ms	~5%

CodeC++

```
1 class MyCircularQueue {
2 private:
3 // i will implement it as a linked list here is my node structure
4 struct Node {
5     int val;
6     Node* next;
7     Node(int val){
8         this->val = val;
```

View more

More challenges

641. Design Circular Deque

1670. Design Front Middle Back Queue

Write your notes here

Select related tags0/5

✓

Code

C++Auto

```
41
42 // this is how i link the circular queue, after we are done adding the node
43 // i make the new tail point back to the head making a circle
44 tail->next = head;
45 size++;
46 return true;
47 }
48
49 bool deQueue() {
50     // if the queue is not empty i move the head forward and delete the node in front
51     if(isEmpty()){
52         return false;
53     }
54
55     Node* temp = head;
56
57     // this line is for handling an edge case where there is only one node in the queue
58     // so it would be pointing to itself, so when the head tries to move forward it loops
59     // back to itself, thus when the node gets deleted head become a dangling pointer
60     // setting tail to null breaks the loop and allows head to move to being null insted
61     tail->next = nullptr;
62
63     head = head->next;
64     delete temp;
65     size--;
66
67     // if there was just one node and it gets deleted tail will be a null
68     // and i when i try to accsess tail->next later it seg faults so we need to
69     // just set to null and exsist the function
70     if(head = nullptr){
71         tail = nullptr;
72         return true;
73     }
74
75     tail->next = head;
76     return true;
77 }
78
79 // rest of the code is pretty straight forward
80 int Front() {
81     if(isEmpty()){
82         return -1;
83     }
84     return head->val;
```

--INSERT--

Saved

Ln 60, Col 39

Testcase

Test Result

Accepted 59 / 59 testcases passed

yousab21 submitted at Jun 29, 2026 19:32

Analysis Solution



Code | C++

```
1 class MyCircularQueue {
2 private:
3 // i will implement it as a linked list here is my node structure
4 struct Node {
5     int val;
6     Node* next;
7     Node(int val){
8         this->val = val;
9     }
10 }
11 ...
12 }
```

View more

More challenges

641. Design Circular Deque 1670. Design Front Middle Back Queue

Write your notes here

Select related tags 0/5

```
68 // and i when i try to access tail->next later it seg faults so we need to
69 // just set to null and exsit the function
70 if(head == nullptr){
71     tail = nullptr;
72     return true;
73 }
74
75 tail->next = head;
76 return true;
77 }
78
79 // rest of the code is pretty straight forward
80 int Front() {
81     if(isEmpty()){
82         return -1;
83     }
84     return head->val;
85 }
86
87 int Rear() {
88     if(isEmpty()){
89         return -1;
90     }
91     return tail->val;
92 }
93
94 bool isEmpty() {
95     return (size == 0);
96 }
97
98 bool isFull() {
99     return (size >= capacity);
100 }
101 };
102
103 /**
104 * Your MyCircularQueue object will be instantiated and called as such:
105 * MyCircularQueue* obj = new MyCircularQueue(k);
106 * bool param_1 = obj->enqueue(value);
107 * bool param_2 = obj->dequeue();
108 * int param_3 = obj->front();
109 * int param_4 = obj->rear();
110 * bool param_5 = obj->isEmpty();
111 * bool param_6 = obj->isFull();
112 */
```


Description Accepted Editorial Solutions Submissions

All Submissions

Accepted 294 / 294 testcases passed
yousab21 submitted at Jun 29, 2026 21:27

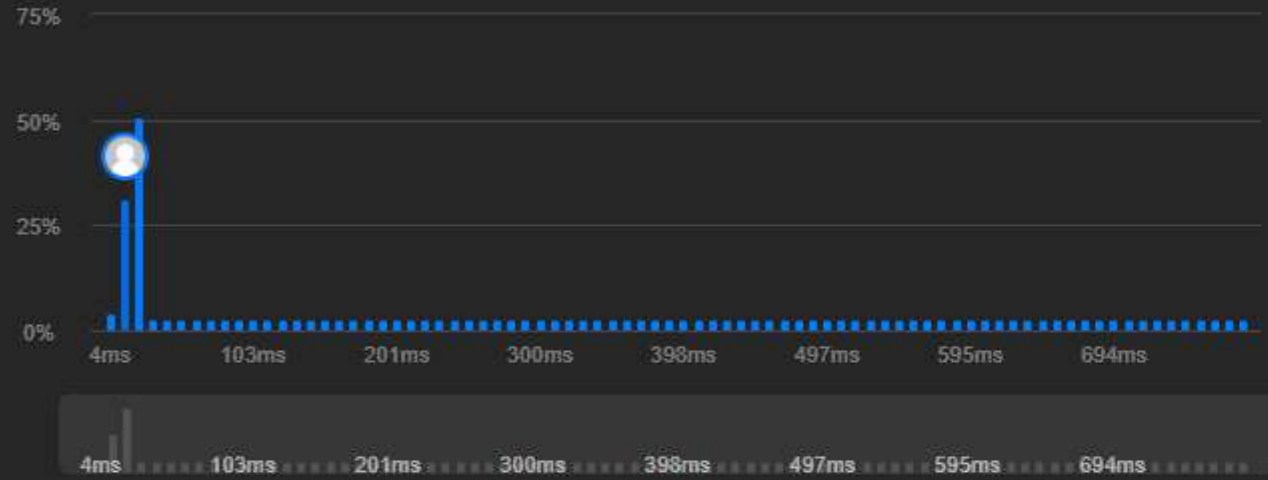
Analysis Solution

Runtime

15 ms | Beats 92.17%

Memory

18.03 MB | Beats 12.20%



Code C++

```
1 // the brute force solution here is to run 4 nested loops (i think it will a
2 // my solution is to take use of the two pointer system we used for the class
3 // and then work out way up the complexty ladder to 3sum then 4sum
4
5 class Solution {
6 private:
7 // here is classic 2sum using 2 pointer i solved this exact problem before s
8 vector<vector<int>> twoSum(vector<int>& array, int start, long long target){
```

View more

More challenges

- 15. 3Sum
- 454. 4Sum II
- 1995. Count Special Quadruplets

Write your notes here

Select related tags

0/5

Code

C++ Auto

```
1 // the brute force solution here is to run 4 nested loops (i think it will accept but i can think of a better one)
2 // my solution is to take use of the two pointer system we used for the classic 2sum
3 // and then work out way up the complexty ladder to 3sum then 4sum
4
5 class Solution {
6 private:
7 // here is classic 2sum using 2 pointer i solved this exact problem before so i wont explain the code
8 vector<vector<int>> twoSum(vector<int>& array, int start, long long target){
9     // i just modified it to start from the index i pass to it insted of 0
10    int left = start;
11
12    //classic 2sum
13    vector<vector<int>> matches;
14    int right = array.size() - 1;
15    vector<int> pair;
16    while(left < right){
17        long long sum = array[left] + array[right];
18        if(sum == target){
19            pair.push_back(array[left]);
20            pair.push_back(array[right]);
21            matches.push_back(pair);
22            pair.clear();
23
24            while(left < right && array[left] == array[left+1]) left++;
25            while(left < right && array[right] == array[right-1]) right--;
26            left++;
27            right--;
28        }
29        else if(sum > target){
30            right--;
31        }
32        else if(sum < target){
33            left++;
34        }
35    }
36    return matches;
37 }
38
39 // now we use the 2sum algorithm to solve 3sum finding all triplets to a set target
40 vector<vector<int>> threeSum(vector<int>& array, int start, long long target){
41     // leave at least a pair ofg elements for 2sum to work with
42     int end = array.size() - 2;
43
44     vector<vector<int>> matches;
```

--INSERT--

Saved

Ln 19, Col 41

Testcase Test Result

Problem List

< > 🔍

Description

Accepted ×

Editorial

Solutions

Submissions

All Submissions

Accepted

294 / 294 testcases passed

Analysis

Solution

yousab21

submitted at Jun 29, 2026 21:27

Runtime

15 ms | Beats 92.17%

Memory

18.03 MB | Beats 12.20%

Code

C++

```
1 // the brute force solution here is to run 4 nested loops (i think it will a
2 // my soluton is to take use of the two pointer system we used for the class
3 // and then work out way up the complexty ladder to 3sum then 4sum
4
5 class Solution {
6 private:
7 // here is classic 2sum using 2 pointer i solved this exact problem before s
8 vector<vector<int>> twoSum(vector<int>& array, int start, long long target){

```

View more

More challenges

15. 3Sum

454. 4Sum II

1995. Count Special Quadruplets

Write your notes here

Select related tags 0/5

👍

👎

👏

👍

👎

👏

Submit

📄

🔍

Code

C++ Auto

```
40 // leave at least a pair of elements for 2sum to work with
41 int end = array.size() - 2;
42
43 vector<vector<int>> matches;
44 vector<int> triplet;
45 // here is the interesting part, i walk along every element in the array
46 // and i run 2sum with a target - element with gets me all the pairs that add up to the highr sum of the triplet
47 for(int i = start; i < end; i++){
48     vector<vector<int>> pairs = twoSum(array, i+1, (target-array[i]));
49     // if we found good pairs we extract the vector of vectors into separate entries with the matching element
50     if(!pairs.empty()){
51         for (int j = 0 ; j < pairs.size(); j++){
52             triplet.push_back(array[i]);
53             triplet.insert(triplet.end(), pairs[j].begin(), pairs[j].end());
54             matches.push_back(triplet);
55             triplet.clear();
56         }
57     }
58     // skip duplicate elements
59     while(i + 1 < end && array[i] == array[i+1]) i++;
60 }
61 // return the triplets that match the target
62 return matches;
63 }
64
65 public:
66 // i then use the 3sum algorithm to walk my way into 4sum
67 vector<vector<int>> fourSum(vector<int>& nums, long long target) {
68     vector<vector<int>> matches;
69
70     // if the array have less than 4 elements just exit immediately
71     if(nums.size() < 4){
72         return matches;
73     }
74
75     // here i sort the array for the whole problem and then pass by reference to 2sum and 3sum instead of sorting it
76     // inside every algorithm
77     sort(nums.begin(), nums.end());
78     // leave at least 3 elements for the 3sum algorithm
79     int end = nums.size() - 3;
80
81     vector<int> quadruplet;
82
83     // here is the even more interesting part i walk through the array and

```

--INSERT--

Saved

Ln 15, Col 23

Testcase

Test Result

[Description](#) | [Accepted](#)  | [Editorial](#) | [Solutions](#) | [Submissions](#)

[← All Submissions](#) 

Accepted 294 / 294 testcases passed

 [Analysis](#)  [Solution](#)

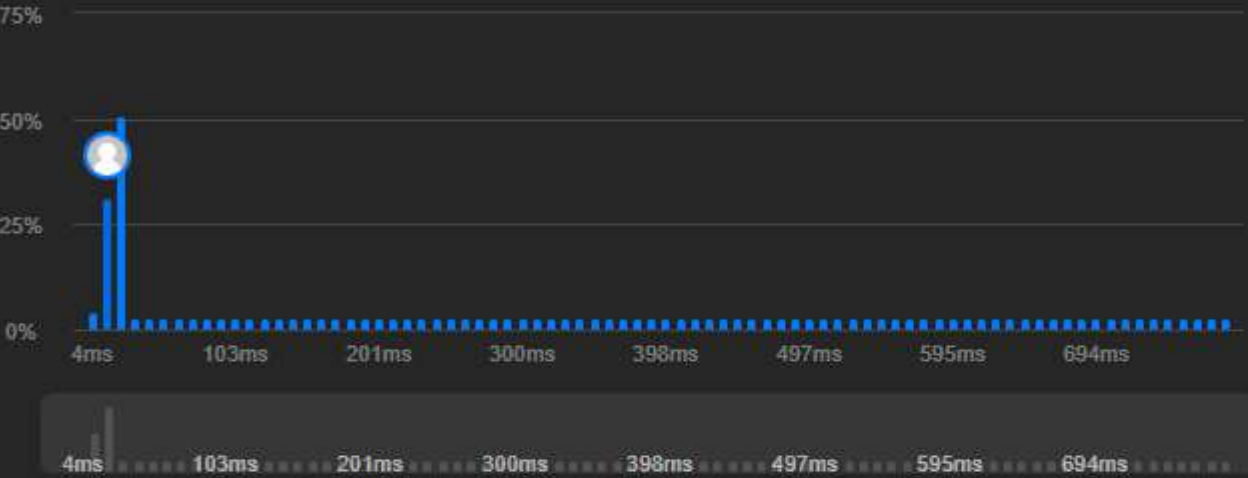
 **yousab21** submitted at Jun 29, 2026 21:27

 **Runtime**

15 ms | Beats **92.17%** 

 **Memory**

18.03 MB | Beats **12.20%**



75%
50%
25%
0%

4ms 103ms 201ms 300ms 398ms 497ms 595ms 694ms

Code | C++

```
1 // the brute force solution here is to run 4 nested loops (i think it will a
2 // my solution is to take use of the two pointer system we used for the class
3 // and then work out way up the complexty ladder to 3sum then 4sum
4
5 class Solution {
6 private:
7 // here is classic 2sum using 2 pointer i solved this exact problem before s
8 vector<vector<int>> twoSum(vector<int>& array, int start, long long target){
    View more
```

More challenges

• 15. 3Sum

• 454. 4Sum II

• 1995. Count Special Quadruplets

Write your notes here

Select related tags 0/5

 **Code**  

C++   Auto     

```
64 }
65
66 public:
67 // i then use the 3sum algorithm to walk my way into 4sum
68 vector<vector<int>> fourSum(vector<int>& nums, long long target) {
69     vector<vector<int>> matches;
70
71     // if the array have less than 4 elements just exit immediatly
72     if(nums.size() < 4){
73         return matches;
74     }
75
76     // here i sort the array for the whole problem and the n pass by refrence to 2sum and 3sum insted of sorting it
77     // inside every algorithm
78     sort(nums.begin(), nums.end());
79     // leave at least 3 elemetns for the 3sum algorithm
80     int end = nums.size() - 3;
81
82     vector<int> quadruplet;
83
84     // here is the even more intresting part,i walk throgh tge array and
85     // run my 3sum algorithm with target - element which gives my all the triplets that sum up to the actual target
86     for(int i = 0; i < end; i++){
87         vector<vector<int>> triplets = threeSum(nums, i+1, (target-nums[i]));
88         // if we find any triplets again unpace the vecctor of vectors same as the 3sum algorithm but with triplets
89         if(!triplets.empty()){
90             for (int j = 0 ; j < triplets.size(); j++){
91                 quadruplet.push_back(nums[i]);
92                 quadruplet.insert(quadruplet.end(), triplets[j].begin(), triplets[j].end());
93                 matches.push_back(quadruplet);
94                 quadruplet.clear();
95             }
96         }
97         //skip duplicates
98         while(i+1 < end && nums[i] == nums[i+1]) i++;
99     }
100     //finally we have out qdruplets and we can return
101     return matches;
102 }
103 };
```

--INSERT--

Saved

 [Testcase](#)  [Test Result](#)

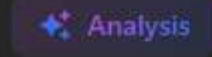
Ln 61, Col 6

Expand Panel **Ctrl** **J**   Editorial  Solutions  Submissions

All Submissions 

Accepted 25 / 25 testcases passed

 **yousab21** submitted at Jun 30, 2026 21:26

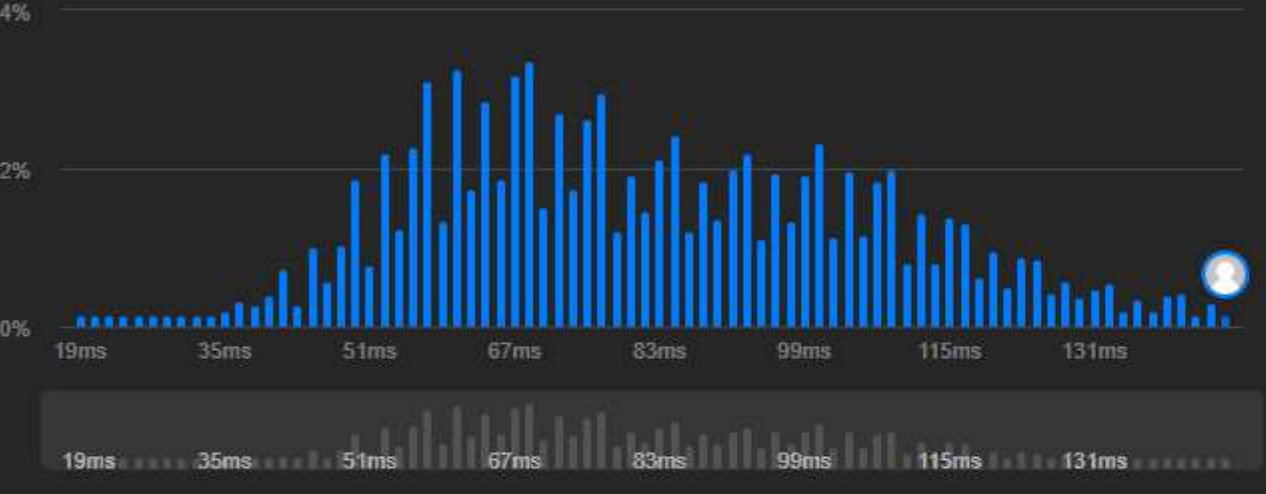
 Analysis  Solution

 Runtime

217 ms | Beats 5.00%

 Memory

194.12 MB | Beats 5.11%



4%
2%
0%

19ms 35ms 51ms 67ms 83ms 99ms 115ms 131ms

19ms 35ms 51ms 67ms 83ms 99ms 115ms 131ms

Code | C++

```
1 // took some time to figure out what an lru chche is but we got there
2 // the solution looks somewhat complex but it is actually really simple
3 class LRUCache {
4 private:
5     // this doubly linked list is how i will track the age of the entries
6     // newer entries will be towareds the tail older ones towards the head
7     // i tried using a queue for this but we need to access random elements
8     // in O(1) so doubly linkeds list with saved pointers in a map it is
```

 View more

More challenges

 460. LFU Cache

 588. Design In-Memory File System

 604. Design Compressed String Iterator

Write your notes here

Select related tags 0/5

Code

C++  Auto     

```
1 // took some time to figure out what an lru chche is but we got there
2 // the solution looks somewhat complex but it is actually really simple
3 class LRUCache {
4 private:
5     // this doubly linked list is how i will track the age of the entries
6     // newer entries will be towareds the tail older ones towards the head
7     // i tried using a queue for this but we need to access random elements from the middle
8     // in O(1) so doubly linkeds list with saved pointers in a map it is
9 struct Node{
10     int key;
11     int val;
12     Node* next = nullptr;
13     Node* prev = nullptr;
14
15     Node(int key, int val){
16         this->key = key;
17         this->val = val;
18     }
19 };
20
21 // the whole "get in O(1)" screams hashmap this one saves pointer for entries in the list by there key
22 map<int,Node*> cache;
23
24 // code was getting a bit complex inside proble function so i pulled out there helper functions
25
26 // this takes a key and removes its entry node from the linked list and map
27 // also i pad the list with a dummy node incase we remove the head
28 void remove(int key){
29     Node* temp = cache[key];
30
31     // if we are removing the tail, just move tail back
32     if(temp == tail){
33         tail = temp->prev;
34         tail->next = nullptr;
35         delete (temp);
36         cache.erase(key);
37         size--;
38         return;
39     }
40
41     temp->next->prev = temp->prev;
42     temp->prev->next = temp->next;
43
44     size--;
```

--NORMAL--

Saved

Ln 115, Col 9

 Testcase  Test Result

Problem List

DescriptionAcceptedEditorialSolutionsSubmissions

All Submissions

Accepted25 / 25 testcases passed

yousab21 submitted at Jun 30, 2026 21:26

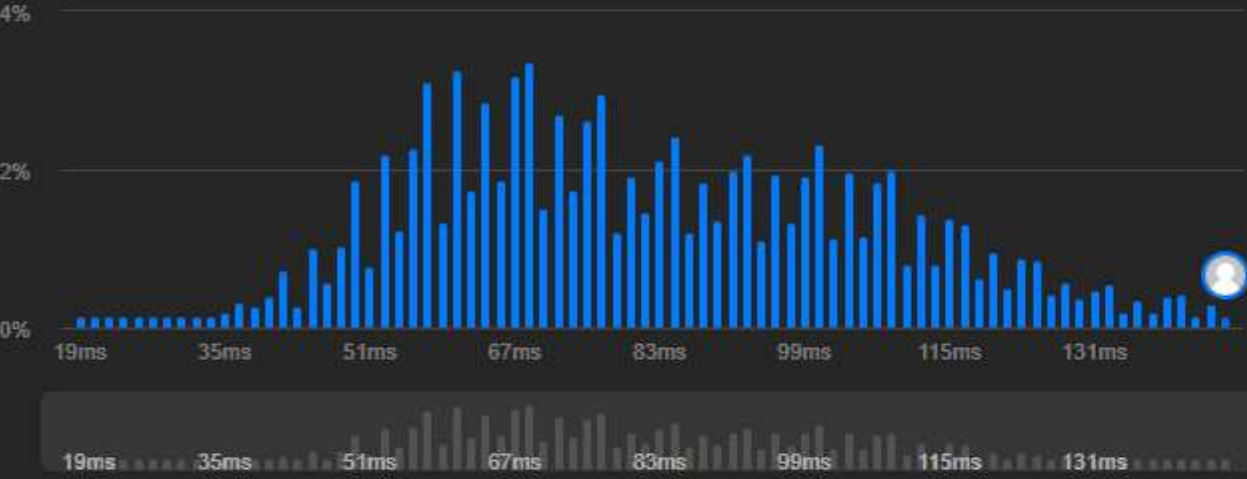
AnalysisSolution

Runtime

217 ms | Beats 5.00%

Memory

194.12 MB | Beats 5.11%



Code | C++

```
1 // took some time to figure out what an lru chche is but we got there
2 // the solution looks somewhat complex but it is actually really simple
3 class LRUCache {
4 private:
5     // this doubly linked list is how i will track the age of the entries
6     // newer entries will be towareds the tail older ones towards the head
7     // i tried using a queue for this but we need to accsess random elements
8     // in O(1) so doubly linkeds list with saved pointers in a map it is
```

View more

More challenges

460. LFU Cache

588. Design In-Memory File System

604. Design Compressed String Iterator

Write your notes here

Select related tags0/5

Code

SubmitCtrlEnter

C++Auto

```
41 temp->next->prev = temp->prev;
42 temp->prev->next = temp->next;
43
44 size--;
45 delete(temp);
46 cache.erase(key);
47 }
48
49 // this adddes a new entry in the end of the linked list and inerts the adress of the node in the map
50 // this is how iam able to remove or update such node in O(1) without traversingf the list
51 void add(int key, int val){
52     Node* newNode = new Node(key, val);
53     // this is just an edge case for the first node
54     // throug the whole runtime head stays as the initial dummy node,
55     // i would be removing nodes from it so no need to move it
56     if(size == 0){
57         head->next = newNode;
58         newNode -> prev = head;
59         tail = newNode;
60         size ++;
61         cache.insert({key, newNode});
62         return;
63     }
64
65     // standard insert into doubly linked list + add to map + update size
66     tail->next = newNode;
67     newNode->prev = tail;
68     tail = newNode;
69     cache.insert({key, newNode});
70     size++;
71 }
72
73 // this return the first element after the dummy node
74 int getOldestKey(){
75     return head->next->key;
76 }
77
78 // this is the heart of the problem
79 // here i use my helper functions to remove the node and push back at the last/newest position
80 // this how "recently used" aspect of the cache works
81 void updateAge(int key){
82     int valTemp = cache[key]->val;
83     remove(key);
84     add(key, valTemp);
85 }
```

--NORMAL--

SavedLn 115, Col 9

TestcaseTest Result

[Description](#) | [Accepted](#)  | [Editorial](#) | [Solutions](#) | [Submissions](#)

[← All Submissions](#) 

Accepted 25 / 25 testcases passed

 **yousab21** submitted at Jun 30, 2026 21:26

[Analysis](#) [Solution](#)

Runtime

217 ms | Beats 5.00%

Memory

194.12 MB | Beats 5.11%



4%
2%
0%

19ms 35ms 51ms 67ms 83ms 99ms 115ms 131ms

19ms 35ms 51ms 67ms 83ms 99ms 115ms 131ms

Code | C++

```
1 // took some time to figure out what an lru chche is but we got there
2 // the solution looks somewhat complex but it is actually really simple
3 class LRUCache {
4 private:
5     // this doubly linked list is how i will track the age of the entries
6     // newer entries will be towareds the tail older ones towards the head
7     // i tried using a queue for this but we need to accsess random elements
8     // in O(1) so doubly linkeds list with saved pointers in a map it is
```

 View more

More challenges

 460. LFU Cache

 588. Design In-Memory File System

 604. Design Compressed String Iterator

Write your notes here

Select related tags 0/5

[Code](#) [Submit](#) [Ctrl](#) [Enter](#)

C++ 

Auto 

```
83         remove(key);
84         add(key, valTemp);
85     }
86
87     Node* head;
88     Node* tail;
89
90     int capacity = 0;
91     int size = 0;
92
93 public:
94     LRUCache(int capacity) {
95         // just set the capacity and add the dummy node
96         this->capacity = capacity;
97         Node* dummy= new Node(-1,-1);
98         head = tail = dummy;
99     }
100
101     int get(int key) {
102         if(size ≤ 0 || !cache.contains(key)){
103             return -1;
104         }
105         // notice how i update the age of the key before returning it
106         int value = cache[key]→val;
107         updateAge(key);
108         return value;
109     }
110
111     void put(int key, int value) {
112         // if the cache already have the key no need to reinsert it just update its value and age
113         if (cache.contains(key)) {
114             updateAge(key);
115             cache[key]→val = value;
116             return;
117         }
118
119         // if the cache still hase space jusy add nodes without removing old ones
120         if(size < capacity){
121             add(key, value);
122             return;
123         }
124         // if we dont have space start removing old nodes
125         else{
126
```

--NORMAL--

Saved

Ln 115, Col 9

 Testcase

 Test Result

Accepted 22 / 22 testcases passed
yousab21 submitted at Jun 30, 2026 23:32

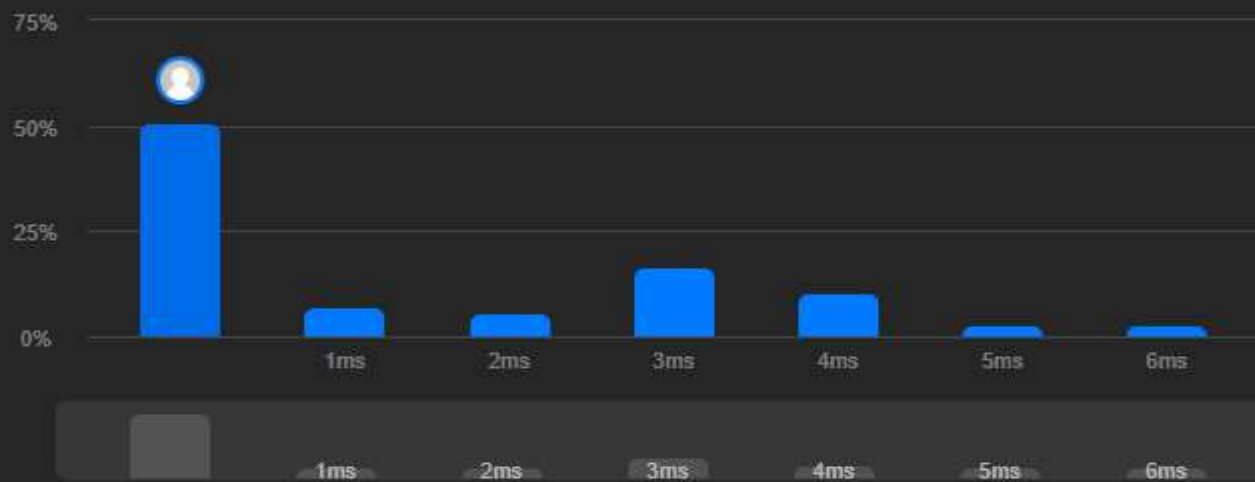
Analysis Solution

Runtime

0 ms | Beats 100.00%

Memory

17.01 MB | Beats 66.22%



Code C++

```
1 class Solution {
2 private:
3 // calculator 9000 pro max code :
4 int doOperation(int num1, int num2, char op){
5     int result;
6     switch (op){
7         case '+':
8             result = num1 + num2;
```

View more

More challenges

224. Basic Calculator

282. Expression Add Operators

Write your notes here

Select related tags

0/5

Code

C++ Auto

```
1 class Solution {
2 private:
3 // calculator 9000 pro max code :
4 int doOperation(int num1, int num2, char op){
5     int result;
6     switch (op){
7         case '+':
8             result = num1 + num2;
9             break;
10        case '-':
11            result = num1 - num2;
12            break;
13        case '*':
14            result = num1 * num2;
15            break;
16        case '/':
17            result = num1 / num2;
18            break;
19    }
20    return result;
21 }
22 // the reason i pass the whole string
23 // insted of casting the first letter into a char and pass said char is pretty sneaky
24 // consider negative numbers (like -16) here the first char would be '-' so it would think the token is an op
25 // this is why i do the checking like this
26 bool isOperation(string token){
27     return (token.size() == 1 && (token[0] == '+' || token[0] == '-' || token[0] == '*' || token[0] == '/' ));
28 }
29
30 public:
31 int evalRPN(vector<string>& tokens) {
32     // we did a very simmiler problem in stack week so this was kinda obvios
33     stack<int> nums;
34
35     // i run throug the tokens one by one, if it is a number i push to stake
36     // if it is an operation i do it and push the result to the stack
37     for(int i = 0; i < tokens.size(); i++){
38         if(!isOperation(tokens[i])){
39             // stoi is string to int
40             nums.push(stoi(tokens[i]));
41         }
42         else if(isOperation(tokens[i])){
43             // nums 1 and 2 are in the order to obey rpn rules in deviation and subtraction
44             int num2 = nums.top();
```

--INSERT--

Saved

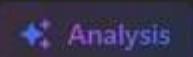
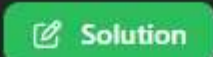
Ln 56, Col 6

Testcase Test Result

[Description](#) | [Accepted](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

[All Submissions](#)

Accepted 22 / 22 testcases passed

 **yousab21** submitted at Jun 30, 2026 23:32

Runtime

0 ms | Beats 100.00%

Memory

17.01 MB | Beats 66.22%

Code | C++

```
1 class Solution {
2 private:
3 // calculator 9000 pro max code :
4 int doOperation(int num1, int num2, char op){
5     int result;
6     switch (op){
7         case '+':
8             result = num1 + num2;
9     }
10 }
```

[View more](#)

More challenges

224. Basic Calculator

282. Expression Add Operators

Write your notes here

Select related tags 0/5

Code

C++  Auto     

```
18         break;
19     }
20     return result;
21 }
22 // the reason i pass the whole string
23 // insted of casting the first letter into a char and pass said char is pretty sneaky
24 // consider negative numbers (like -16) here the first char would be '-' so it would think the token is an op
25 // this is why i do the checking like this
26 bool isOperation(string token){
27     return (token.size() == 1 && (token[0] == '+' || token[0] == '-' || token[0] == '*' || token[0] == '/' ));
28 }
29
30 public:
31 int evalRPN(vector<string>& tokens) {
32     // we did a very simmler problem in stack week so this was kinda obvios
33     stack<int> nums;
34
35     // i run throug the tokens one by one, if it is a number i push to stake
36     // if it is an operation i do it and push the result to the stack
37     for(int i = 0; i < tokens.size(); i++){
38         if(!isOperation(tokens[i])){
39             // stoi is string to int
40             nums.push(stoi(tokens[i]));
41         }
42         else if(isOperation(tokens[i])){
43             // nums 1 and 2 are in the order to obey rpn rules in deviation and subtraction
44             int num2 = nums.top();
45             nums.pop();
46             int num1 = nums.top();
47             nums.pop();
48
49             char op = tokens[i][0];
50
51             int result = doOperation(num1, num2, op);
52             nums.push(result);
53         }
54     }
55     return nums.top();
56 }
57 };
```

--INSERT--

Saved

Ln 56, Col 6

 Testcase

 Test Result